



Ghulam Ishaq Khan Institute of Engineering Sciences and Technology

Faculty of Computer Science and Engineering

CE222 – Computer Organization and Assembly Language

VERA-16

A 2-Address 16-bit Processor with General-Purpose Register File

Semester Processor Design Project

Submitted by:

Usman Azhar
(2024653)

Reyan Kashif
(2024538)

Muhammad Saim
(2024453)

Submitted to Prof. Dr. Ghulam Abbas

Faculty of Cyber Security

Submission Date: April 20, 2026

Abstract

VERA-16 is a 16-bit processor created specifically for the project of Computer Organization and Assembly Language (COAL). The goal of this design was to alleviate a major chokepoint present in the Mano Machine. In specific a single accumulator design in which all of the arithmetic operations are carried out through a single register.

VERA-16 substitutes the need of a single accumulator with a register file of eight general purpose registers (R0-R7) consenting any of the two registers to act as operands in an individual instruction. This 2-address structure of VERA-16 eradicates the necessity to route every computation through a fixed register which is a substantial departure from the conventional Mano Machine's design.

The processor employs a 16-bit instructions word segregated into a 5-bit op-code, a 2-bit addressing mode, a 3-bit destination register and a 6-bit source field. This layout of instructions permits VERA-16 to support 33 instructions divided into the distinct categories consisting of arithmetic, logic, shift, memory, program control and I/O as well as the four addressing modes and an interrupt mechanism.

The design differs greatly from the conventional Mano Machine in op-code breath, register structure, instruction format, ALU competence and addressing versatility. The report is accompanied by the development of software simulator.

1. Introduction

The Mano Machine represents a conventional 1-address accumulator based design. Every arithmetic operation pursues the same pattern. The accumulator (AC) holds a single operand and the other is read from the memory into the data register (DR). The operation is performed and the result is deposited back in AC. While polished in its minimalistic behavior this approach introduces an architectural chokepoint when building real programs. Each computation necessitates at least one memory fetch to load the second operand, and results must be returned and stored back to memory before they may be reused in some other instruction.

After examining the Mano Machine thoroughly our group identified a significant drawback via micro operation sequences. For a simple instruction like adding two numbers Mano Machine demands fetch the instruction retrieve the operand from memory perform the addition operation on the two and then the store the result in AC. It cannot be passed on directly to the next operation without another memory contact. This is the accumulator chokepoint.

This bottleneck of Mano Machine was eliminated by this design of VERA-16. The name reflects the architectural aim behind it. A Versatile Extended Register Architecture where both the source and destination of each of the instruction are designated as addresses within a file of eight general-purpose registers. Because of the reason that operands are stored in registers rather than in memory arithmetic operations are significantly faster. A register access takes up just a single clock cycle whereas memory access takes several cycles.

The design follows the framework of Morris Mano's System Architecture tailoring and expanding the concepts to reflect a modern register-based approach. Whereas the conventional Mano Machine employs a 3-bit opcode with only 8 memory reference instructions. He VERA-16 uses a 5-bit opcode with the provided 32 instruction slots and 33 distinct operations that are specified across five instruction categories.

2. Design Philosophy and Innovation

The primary design decision in VERA-16 is the substitution of single accumulator with a general purpose register file. VERA-16 provides eight 16-bit registers (R0 through R7) that are completely interchangeable and any of the registers can be a source of operand and destination for a result.

Second, VERA-16 uses [OP-CODE (5) | MODE (2) | DST (3) | SRC (6)]. The DST field determines which of the available register receives the result. The SRC field indicated the second operand. This is why VERA-16 is a 2-address architecture. Each of the instructions in VERA-16 contains two register addresses.

The third substantial novelty in VERA-16 is the increased op-code space. With 5 bits allocated to the op-code, VERA-16 can express up to 32 distinct instruction types compared to the Mano Machine's 8 instruction types. This allows VERA-16 to contain a complete instruction set like MUL, shift operations, compare, I/O skip instructions and register-reference as first-class instructions eliminating the need for extensive programs.

The fourth novelty of VERA-16 is the 2-bit MODE field, which provides the VERA-16 four addressing modes that are register, immediate, direct and register-indirect. So, this addition of immediate mode allows small constants to be encoded directly into the instruction word via LDI, and for full memory access, register-indirect mode allows the register file to act as a pointer.

Fifth VERA-16's register file architecture eliminates the long memory reference chains the sequence counter is reduced from 4-bit to 3-bit going through T0 to T7 (8 Cycles) that is sufficient for all 33 instructions while simplifying the CU hardware.

The 16-bit word size was intentionally chosen to as the field allocation inside those 16-bits is $5+2+3+6 = 16$ bits that are dedicated to control, mode and register fields, which opposed to $1+3+12 = 16$ bits as in the Mano Machine.

3. Word Size, Memory, and Register Organization

3.1 Word Size

VERA-16 uses a 16-bit word size. All registers sustain 16-bit values. The memory is ordered as 16-bit words. ALU performs 16-bit operations. Data values range from 0 to 65,535 for unsigned integers, or from $-32,768$ to $+32,767$ for signed 2's complement depiction. The 16-bit word size was chosen to maintain direct correspondence with the Mano Machine while how a fundamentally different architecture can be achieved within the same word width.

3.2 Memory Organization

VERA-16 uses a 4096×16 RAM. Memory has 4096 address locations, each storing individual 16-bit word. The address space requires 12-bit addresses ($2^{12} = 4096$). Since VERA-16 encodes addresses in registers (which are 16 bits wide), the full 4096-word address space is accessible through register-indirect addressing. Direct address encoding within the instruction word is limited by the SRC field size (6 bits = addresses 0–63), which is why LDM and STM use register indirect addressing as the primary memory access method.

Memory is divided into two logical zones. Addresses 0 through 511 constitute the program zone holding instructions. Addresses 512 through 4095 constitute the data zone. Address 0 is reserved for the interruption save location, as in Mano Machine's interruption mechanism.

3.3 Register Organization

VERA-16 uses the following registers. The register file (R0–R7) forms the core of the architecture, replacing the single AC of the Mano Machine. The remaining registers serve internal CPU functions and are not directly addressable by most instructions.

VERA-16 Register Organization

All registers accessible via the 16-bit common bus

Register	Size (bits)	Purpose					
General-Purpose Register File							
R0	16	General-purpose. I/O convention register (INP loads here, OUT reads from here)					
R1	16	General-purpose					
R2	16	General-purpose					
R3	16	General-purpose					
R4	16	General-purpose					
R5	16	General-purpose					
R6	16	General-purpose					
R7	16	General-purpose					
Control and Internal Registers							
PC	12	Program Counter — holds address of next instruction to fetch. Incremented at T1.					
IR	16	Instruction Register — holds the current instruction. Fields decoded at T2 by control unit.					
AR	12	Address Register — holds memory address during fetch and memory operations (LDM, STM).					
DR	16	Data Register — temporary buffer for memory read results. Loaded from M[AR], then forwarded to Rf[DST].					
TR	16	Temporary Register — used by SWAP instruction and interrupt cycle to save PC value.					
I/O Registers and Flip-Flops							
INPR	8	Input Register — holds 8-bit character received from input device. Read by INP instruction into R0.					
OUTR	8	Output Register — holds 8-bit character sent to output device. Written by OUT instruction from R0[7:0].					
Status Flags and Control Flip-Flops (1-bit each)							
Z	C	N	IEN	FGI	FGO	S	R
Zero flag	Carry flag	Negative flag	Interrupt enable	Input ready	Output ready	Start/Stop	Interrupt request

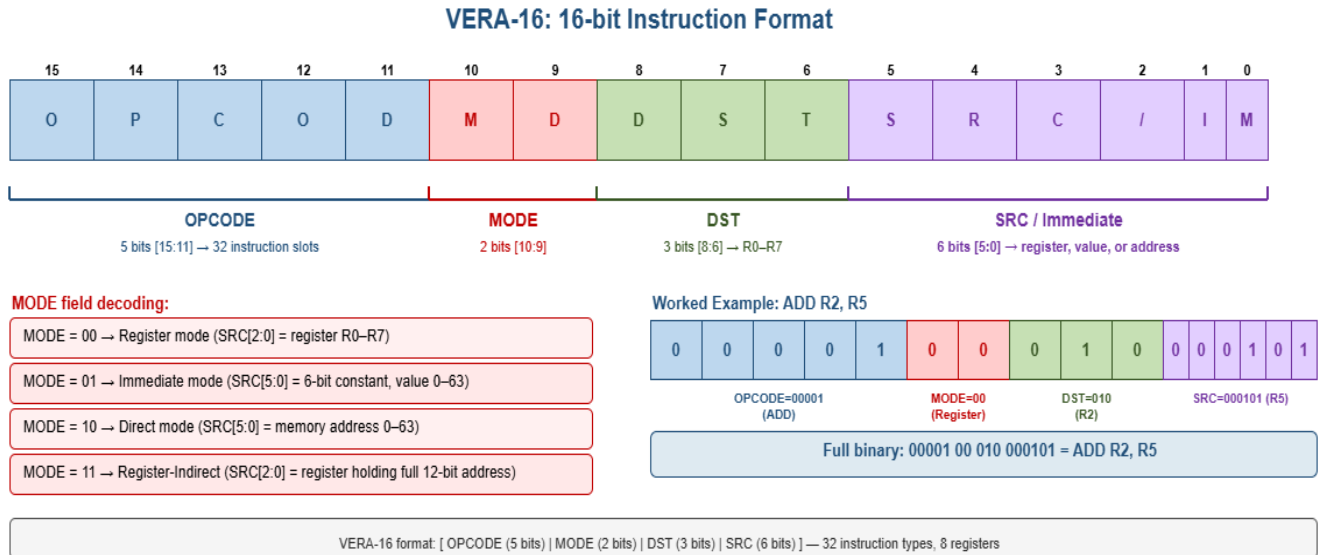
3.4 Flip-Flops and Status Flags

The following status flags and control flip-flops are used to track processor state and control instruction flow. These are distinct from the general-purpose register file.

Flag / FF	Size (bits)	Purpose
Z (Zero)	1	Set to 1 when the ALU result is zero.
C (Carry)	1	Set to 1 when an arithmetic operation produces a carry-out of bit 15, or when a shift shifts out a bit.
N (Negative)	1	Set to 1 when result bit 15 is 1 (negative in two's complement representation).
IEN	1	Interrupt Enable flip-flop. When set to 1, interrupt requests are permitted.
FGI	1	Input Flag. Set to 1 by the input device when new data is ready in INPR.
FGO	1	Output Flag. Set to 1 by the output device when it is ready to accept data from OUTR.
S (Start/Stop)	1	Start-Stop flip-flop. When S=0, the CPU halts execution (same function as Mano Machine).
R (Interrupt)	1	Interrupt request flag. Set when interrupt condition is met (IEN=1 and FGI=1 or FGO=1).
SC (Seq. Counter)	4	Sequence Counter. Counts timing pulses T0 through T5 to sequence microoperations within each instruction cycle.

4. Instruction Format

Every VERA-16 instruction is exactly 16 bits wide. The 16-bit instruction word is divided into four fields. This format is fundamentally different from the Mano Machine's [I | opcode (3) | address (12)] layout. In VERA-16, the 12-bit address field has been replaced by a 5-bit opcode, a 2-bit mode selector and two 3-bit register specifiers, providing significantly more flexibility.



4.1 Worked Example: ADD R2, R5

The instruction ADD R2, R5 means add the value in R5 to the value in R2 and store the result in R2. The binary encoding is as follows:

OPCODE = 00000 (ADD)
 MODE = 00 (register-to-register)
 DST = 010 (R2)
 SRC = 000101 (R5 encoded in lower 3 bits of SRC field)
 Full 16-bit instruction: 00000 00 010 000101

4.2 Comparison with Mano Machine Instruction Format

The Mano Machine uses a 16-bit format [I | opcode (3) | address (12)] that is one address per instruction, with the I bit selecting direct or indirect addressing. VERA-16 replaces this with two register addresses (DST and SRC) and a MODE field, yielding significantly more flexibility at the cost of a reduced direct address range (6 bits instead of 12). The wider opcode field compensates for this by enabling far larger instruction vocabulary.

5. Addressing Modes

VERA-16 supports four addressing modes, controlled by the 2-bit MODE field in bits [10:9] of the instruction word. The Mano Machine supports only two modes (direct and indirect addressing). VERA-16's four modes enable a wider range of programming patterns including immediate constants and efficient register-based pointer operations.

MODE bits	Mode Name	How SRC is Interpreted
00	Register	SRC [2:0] = register number R0–R7. Operand is the value stored in that register.
01	Immediate	SRC [5:0] = 6-bit (values 0–63). Operand is the constant itself.
10	Direct	SRC [5:0] = memory addr. (0–63). Operand is the value stored at that memory location.
11	Register-Indirect	SRC [2:0] = register holding a full 12-bit memory address. Operand is the value at that address.

Note: Immediate mode limitation 6-bit immediate field supports values 0 to 63 only. For larger constants, the programmer must load a value from memory using LDM with a register indirect address. This is a known constraint arising from the decision to allocate 5 bits to the op-code and 5 bits to mode and register fields within the 16-bit instruction word.

6. Instruction Set

VERA-16 implements 33 instructions across 28 op-code encodings. The register-reference group (op-code 11110) uses a single op-code slot with one-hot function bits in IR [8:2] to select seven suboperations. It covers all required categories Arithmetic, Logic, Shift, Data Transfer, Program Control and I/O.

6.1 Complete Instruction Set Table

The table demonstrated below provides the full instruction set specification. Each instruction is enumerated with its binary op-code, mnemonic, operation in register-transfer notation and category.

#	Opcode	Mnemonic	Description	Category
1	00000	ADD	Add registers.	Arithmetic
2	00001	SUB	Subtract registers.	Arithmetic
3	00010	MUL	Multiply registers.	Arithmetic
4	00011	AND	AND memory word to register.	Logic
5	00100	OR	OR memory word to register.	Logic
6	00101	XOR	XOR memory word to register.	Logic
7	00110	MOV	Move register content.	Data Transfer
8	00111	CMP	Compare two registers.	Arithmetic
9	01000	NOT	Complement register.	Logic
10	01001	INC	Increment register.	Arithmetic
11	01010	DEC	Decrement register.	Arithmetic
12	01011	SHL	Shift left logical.	Shift
13	01100	SHR	Shift right logical.	Shift

14	01101	LDI	Load immediate to register.	Data Transfer
15	01110	LDM	Load register from memory.	Data Transfer
16	01111	STM	Store register to memory.	Data Transfer
17	10000	JMP	Branch unconditionally.	Control
18	10001	JZ	Branch if zero.	Control
19	10010	JNZ	Branch if not zero.	Control
20	10011	HLT	Halt computer.	Control
21	10100	INP	Input character to R0.	I/O
22	10101	OUT	Output character from R0.	I/O
23	10110	SKI	Skip on input flag.	I/O
24	10111	SKO	Skip on output flag.	I/O
25	11000	ION	Interrupt on.	I/O
26	11001	IOF	Interrupt off.	I/O
27	11110/IR[8]	CLA	Clear R0 to zero.	Register-Ref.
28	11110/IR[7]	CMR	Complement register.	Register-Ref.
29	11110/IR[6]	CLC	Clear carry flag.	Register-Ref.
30	11110/IR[5]	STC	Set carry flag to 1.	Register-Ref.
31	11110/IR[4]	CLZ	Clear zero flag.	Register-Ref.
32	11110/IR[3]	NOP	No operation Reset sequence counter.	Register-Ref.
33	11110/IR[2]	SWAP	Swap two registers.	Register-Ref.

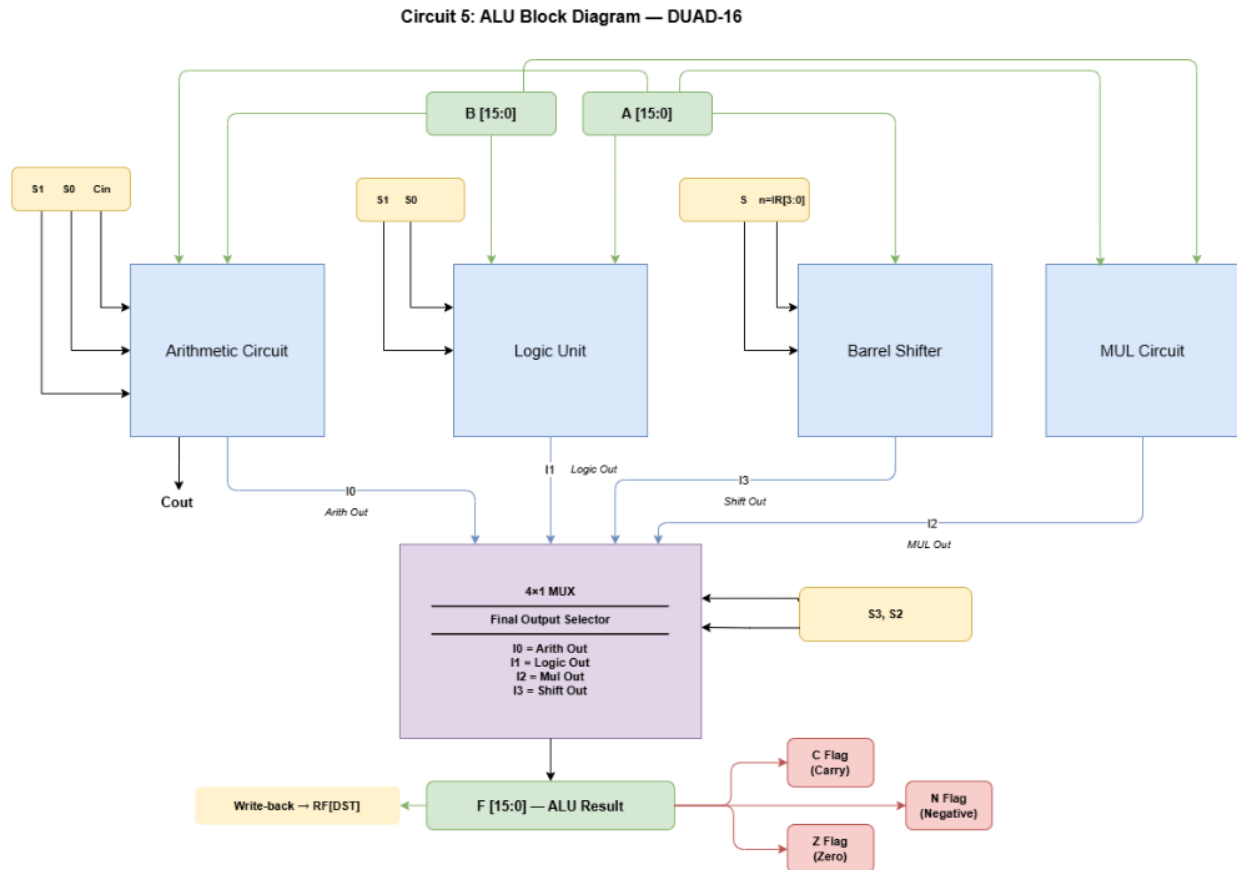
6.2 Register-Reference One-Hot Encoding

The register-reference group uses one-hot bit decoding within bits IR [8:2]. Each bit initiates a distinct sub-operation. This encoding allows seven instructions to share a single opcode slot.

#	Opcode	Mnemonic	Trigger Bit
1	11110/IR [8]	CLA	IR [8] = 1
2	11110/IR [7]	CMR Rd	IR [7] = 1
3	11110/IR [6]	CLC	IR [6] = 1
4	11110/IR [5]	STC	IR [5] = 1
5	11110/IR [4]	CLZ	IR [4] = 1
6	11110/IR [3]	NOP	IR [3] = 1
7	11110/IR [2]	SWAP Rd, Rs	IR [2] = 1

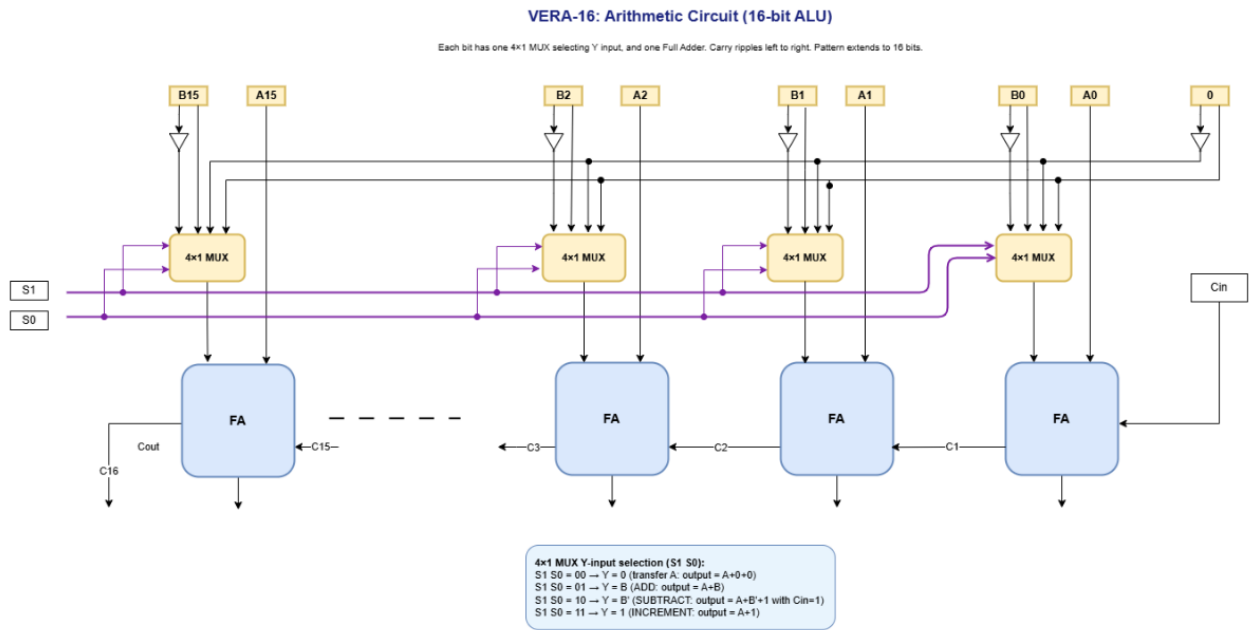
7. ALU Design

The VERA-16 Arithmetic Logic Unit (ALU) is a 16-bit combinational circuit. It consists of four sub-circuits: a 16-bit ripple-carry adder, a 16-bit logic unit, a barrel shifter, and an array multiplier. A 4×1 output multiplexer selects which sub-circuit's result is forwarded.



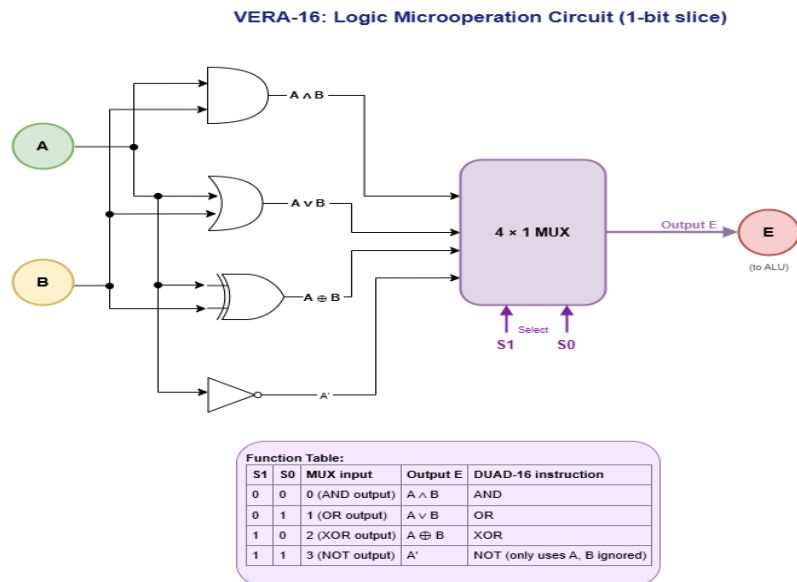
7.1 Arithmetic Circuit

The arithmetic unit is built from 16 full adders chained in a ripple-carry configuration. Each bit slice contains a 4×1 multiplexer that selects the Y-input to the full adder based on control signals S1 and S0. The four MUX inputs are 0 (for transfer/pass-through), B (for addition), B' (for subtraction) and 1 (for increment). The carry-in (Cin) to the least-significant full adder is also controlled, enabling 2's complement subtraction when Cin=1 and B is inverted.



7.2 Logic Unit

The logic unit operates on one-bit slices, replicated 16 times. Each slice takes bits A_i and B_i as inputs, passes them through AND, OR XOR and NOT gates and presents all four results to a 4x1 multiplexer. The select inputs S1 and S0 choose which logic operation appears at output E_i for that bit position.

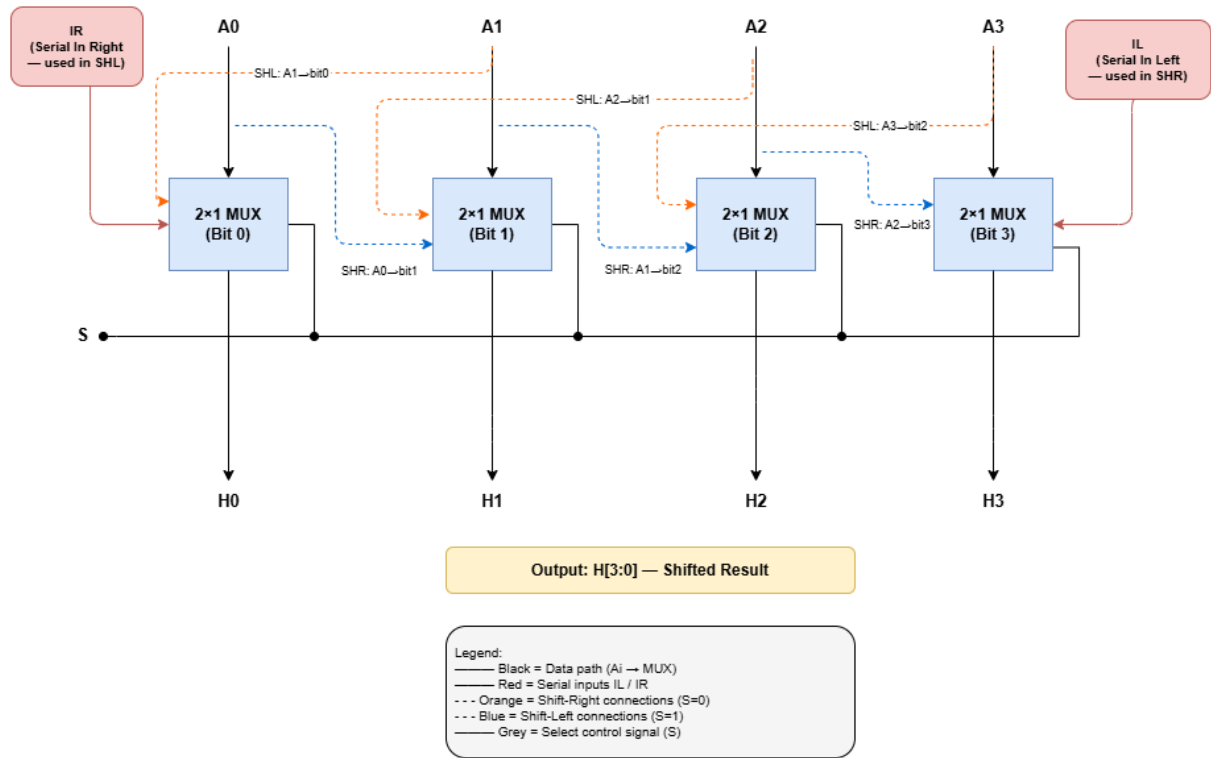


This is 1-bit slice. For VERA-16: replicate this circuit 16 times (one per bit of the 16-bit registers). S1 and S0 are shared control lines across all 16 slices.

7.3 Shift Unit

The shift unit is a barrel shifter capable of shifting the 16-bit input left or right by n positions, where n is encoded in $IR[3:0]$ (values 0–15). Logical shift left (SHL) fills vacated low-order bits with zeros and captures the last shifted-out bit in the Carry flag. Logical shift right (SHR) fills vacated high-order bits with zeros and captures the last shifted-out bit in the Carry flag.

Circuit 4: Shift Microoperation Circuit (4-bit)



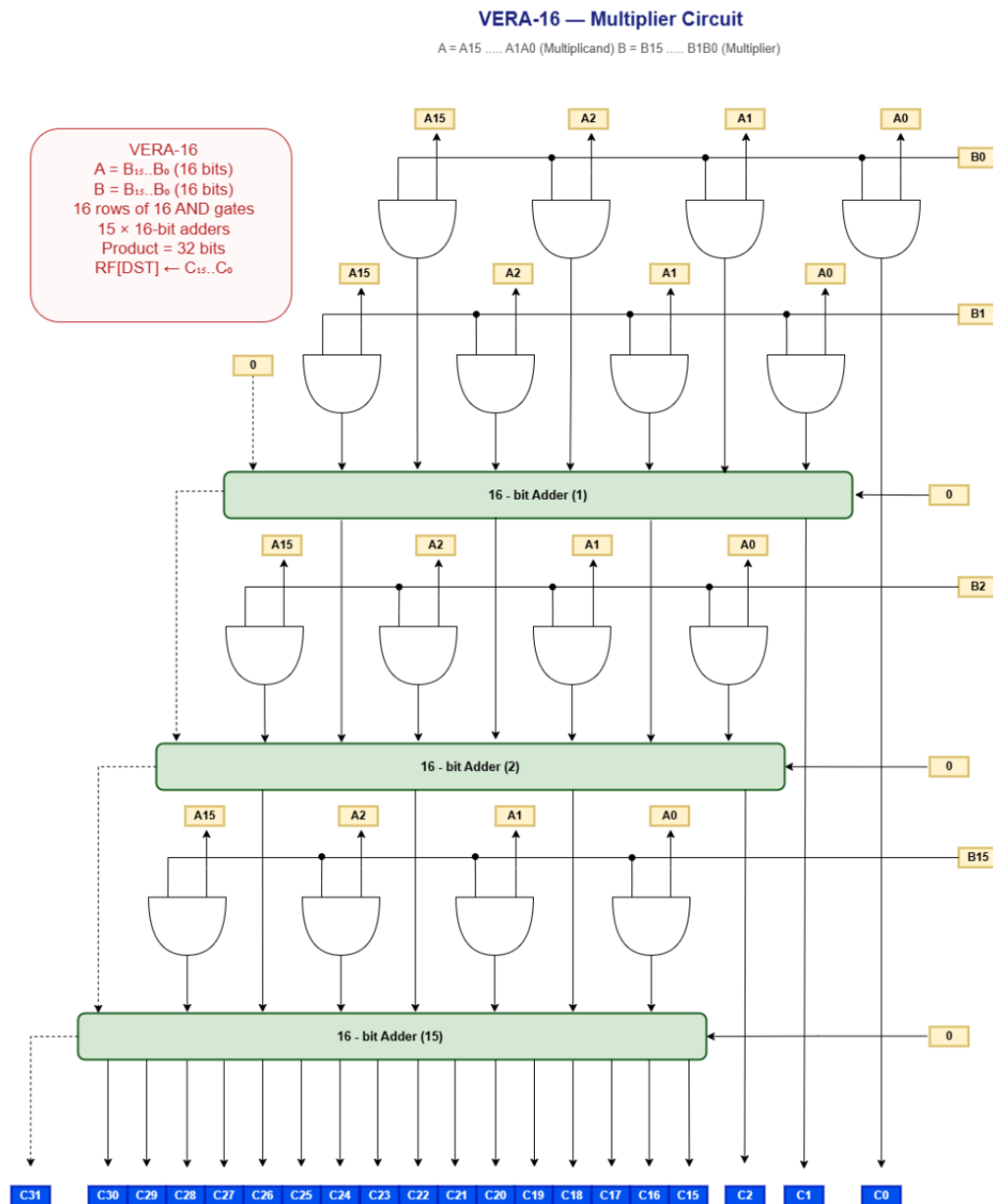
7.4 ALU Control Signal Table

The following table specifies the control signals S_3 , S_2 , S_1 , S_0 , and C_{in} that the Control Unit drives to select each ALU operation. S_3 selects between the arithmetic/shift.

S_3	S_2	S_1	S_0	C_{in}	ALU Function
0	0	0	0	0	ADD
0	0	0	1	1	SUB, CMP
0	0	1	0	0	INC
0	0	1	1	1	DEC
0	1	0	0	0	MUL
0	1	0	1	—	SHL
0	1	1	0	—	SHR
0	1	1	1	—	MOV, LDI, LDM
1	0	0	0	—	AND
1	0	0	1	—	OR
1	0	1	0	—	XOR
1	0	1	1	—	NOT, CMR
1	1	0	0	—	CLA

7.5 MUL Implementation — Array Multiplier

The MUL instruction is implemented using an array multiplier a parallel hardware structure that computes all partial products simultaneously using AND gates then sums them using a chain of adders. In VERA-16 16-bit operands, the pattern has 16 rows of 16 AND gates with 15, 16-bit adders, producing a 32-bit product. The lower 16 bits are stored.



8. Control Unit Design

The Control Unit (CU) of VERA-16 is the primary synchronizing circuit that generates all timing and control signals for each instruction. It interprets the Instruction Register IR and based on the op-code and current timing state prompts every other component in the CPU including the register file selector, ALU, memory and bus multiplexers.

8.1 5-to-32 Opcode Decoder

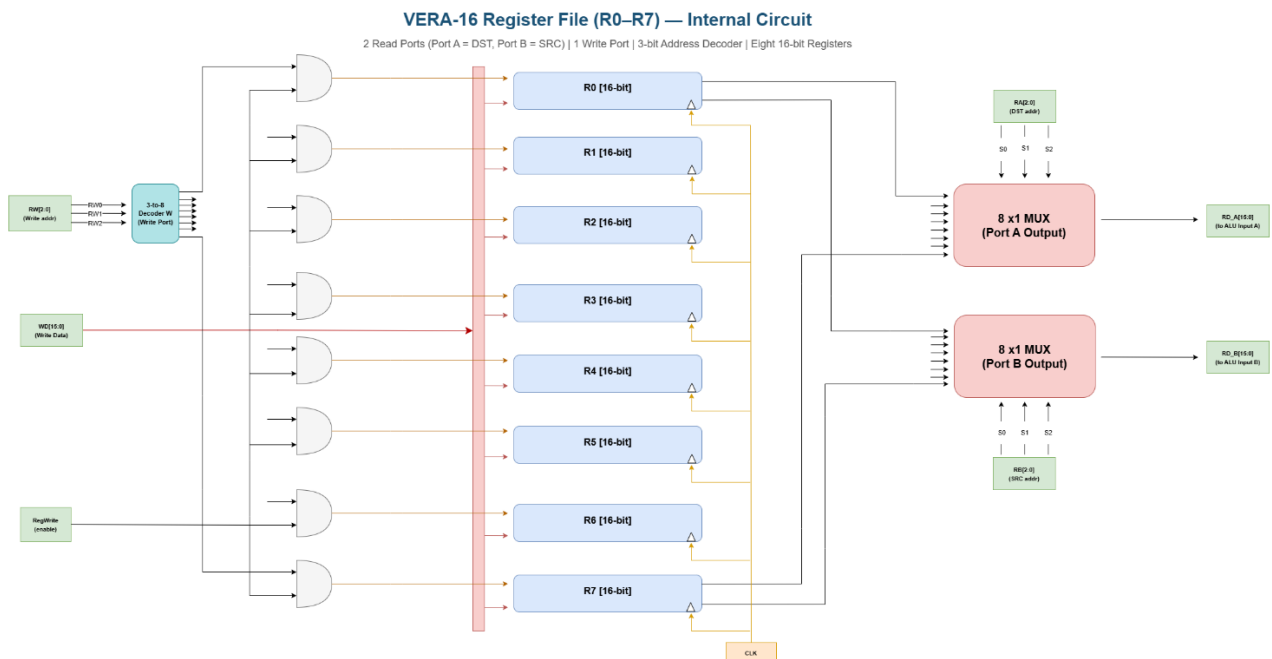
The 5-bit op-code field IR [15:11] is channeled into a 5-to-32 decoder producing 32 output lines D0 to D31. Each active line corresponds to one instruction. This broader decoder is what enables VERA-16's larger instruction set and is the fundamental reason the control logic can support 33 distinct operations.

8.2 Sequence Counter

The sequence counter (SC) is 3-bit counter that induce timing pulses T0 through T7 with a 3-to-8 decoder. The T states: T0, T1, and T2 states are always the fetch and decode phase and are identical for every instruction. T3 onwards represents the execute phase, with the number of cycles fluctuating by instruction type. When the CU asserts $SC \leftarrow 0$, the counter resets to T0, beginning the next instruction cycle.

8.3 Register File Selector

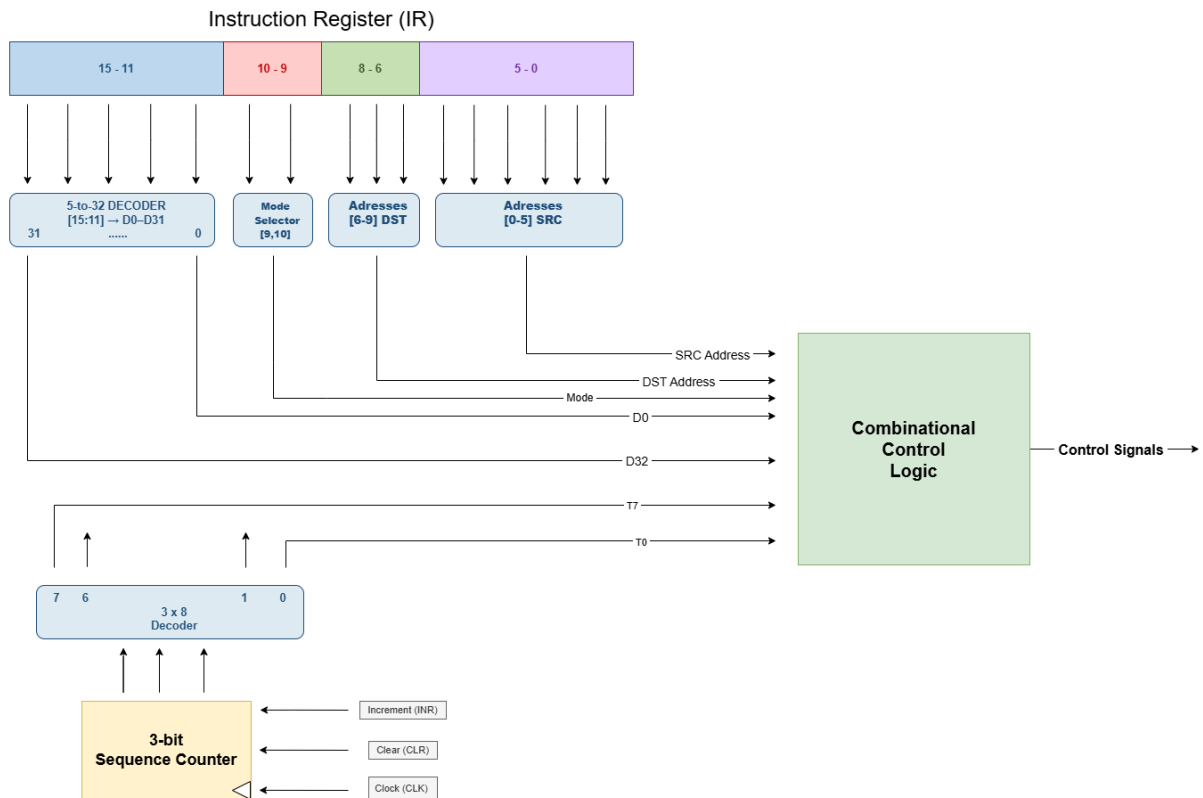
The DST field IR [8:6] and the lower 3 bits of the SRC field IR [5:3] are each decoded by a 3-to-8 decoder, producing read-address-A, read-address-B and write-address signals for the register file. In VERA-16, the register file selector is what makes any register usable as source or destination in any instruction.



8.4 MODE Field Router

The 2-bit MODE field IR[10:9] is decoded by a 2-to-4 decoder and its outputs drive the addressing logic that are Register mode (00), Immediate mode (01), Direct mode (10), Indirect mode (11).

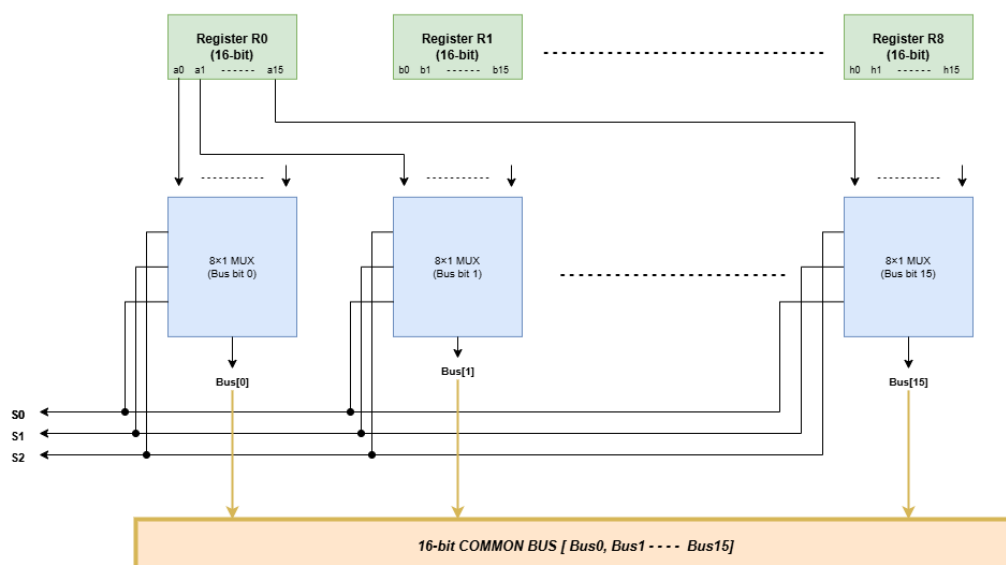
VERA-16 CONTROL UNIT

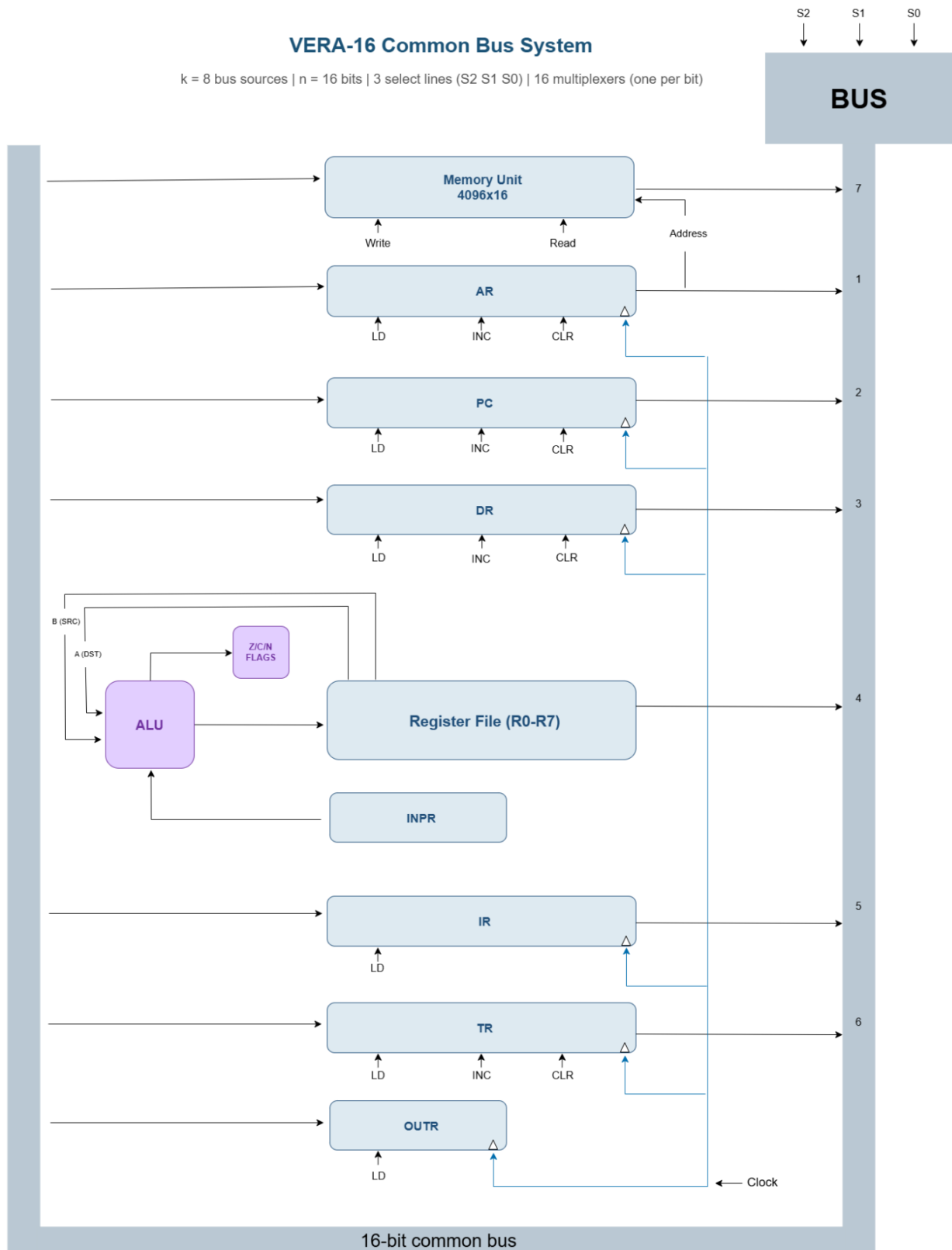


9. Common Bus System

VERA-16 uses a single 16-bit common bus to transfer data between the register file, memory, ALU, PC and IR. The bus is implemented using a series of multiplexers each source connected to the bus contributes one set of MUX inputs. Only one source may drive the bus at any given time and the control unit selects the active source using bus selection signals.

Bus Using MUX — Vera-16

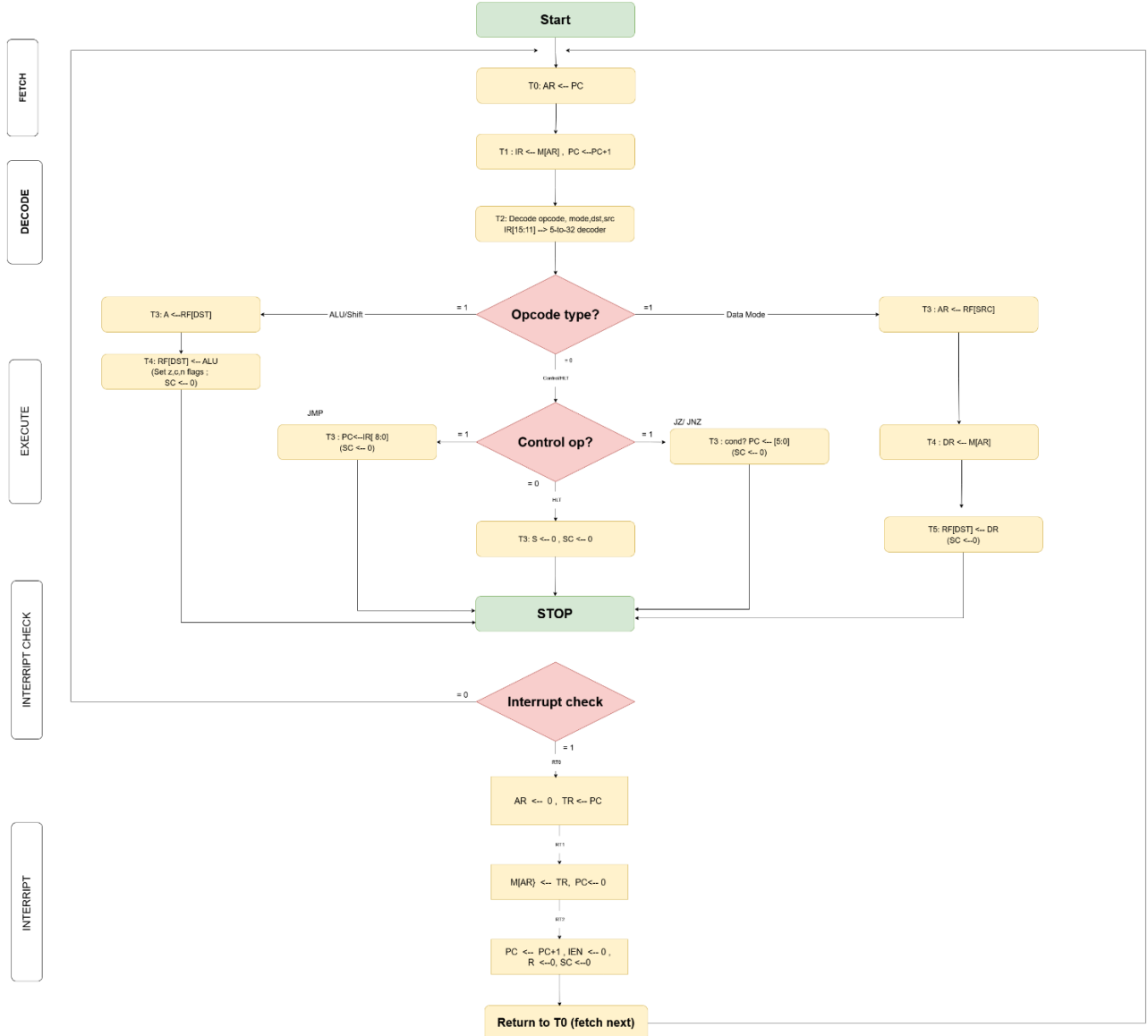




10. Cycle Instruction and Microoperations

Every instruction in VERA-16 follows the same three-phase cycle: Fetch, Decode and Execute. The fetch and decode phases are identical for all instructions. The execute phase varies by instruction type and may take one, two or three timing cycles (T3 only, T3–T4 or T3–T5).

VERA-16 Fetch Decode Execute Cycle



10.1 Fetch and Decode Phase (All Instructions)

The following three microoperations are performed identically at the start of every instruction cycle:

T0: $AR \leftarrow PC$

T1: $IR \leftarrow M[AR], PC \leftarrow PC + 1$

T2: $OPCODE \leftarrow IR[15:11], MODE \leftarrow IR[10:9], DST \leftarrow IR[8:6], SRC \leftarrow IR[5:0]$

5-to-32 decoder activates lines D0–D31 based on OPCODE field

10.2 ADD — Register Addition

T3: $A \leftarrow RF[DST], B \leftarrow RF[SRC]$

T4: $RF[DST] \leftarrow A + B$, set Z/C/N flags, $SC \leftarrow 0$

10.3 SUB — Register Subtraction

T3: $A \leftarrow \text{RF}[\text{DST}], B \leftarrow \text{RF}[\text{SRC}]$
 T4: $\text{RF}[\text{DST}] \leftarrow A - B$, set Z/C/N flags, $\text{SC} \leftarrow 0$

10.4 MUL — Register Multiplication

T3: $A \leftarrow \text{RF}[\text{DST}], B \leftarrow \text{RF}[\text{SRC}]$
 T4: $\text{RF}[\text{DST}] \leftarrow \text{lower 16 bits of } (A \times B)$, set Z/N flags, $\text{SC} \leftarrow 0$

10.5 AND — Bitwise AND

T3: $A \leftarrow \text{RF}[\text{DST}], B \leftarrow \text{RF}[\text{SRC}]$
 T4: $\text{RF}[\text{DST}] \leftarrow A \text{ AND } B$, set Z/N flags, $\text{SC} \leftarrow 0$

10.6 OR — Bitwise OR

T3: $A \leftarrow \text{RF}[\text{DST}], B \leftarrow \text{RF}[\text{SRC}]$
 T4: $\text{RF}[\text{DST}] \leftarrow A \text{ OR } B$, set Z/N flags, $\text{SC} \leftarrow 0$

10.7 XOR — Bitwise XOR

T3: $A \leftarrow \text{RF}[\text{DST}], B \leftarrow \text{RF}[\text{SRC}]$
 T4: $\text{RF}[\text{DST}] \leftarrow A \text{ XOR } B$, set Z/N flags, $\text{SC} \leftarrow 0$

10.8 MOV — Register Copy

T3: $A \leftarrow \text{RF}[\text{DST}], B \leftarrow \text{RF}[\text{SRC}]$
 T4: $\text{RF}[\text{DST}] \leftarrow B$, $\text{SC} \leftarrow 0$

10.9 CMP — Compare (No Write-Back)

T3: $A \leftarrow \text{RF}[\text{DST}], B \leftarrow \text{RF}[\text{SRC}]$
 T4: $\text{temp} \leftarrow A - B$, set Z/C/N flags only, no write-back to register, $\text{SC} \leftarrow 0$

10.10 NOT — Bitwise Complement

T3: $A \leftarrow \text{RF}[\text{DST}]$
 T4: $\text{RF}[\text{DST}] \leftarrow \text{NOT } A$, set Z/N flags, $\text{SC} \leftarrow 0$

10.11 INC — Increment

T3: $A \leftarrow \text{RF}[\text{DST}]$
 T4: $\text{RF}[\text{DST}] \leftarrow A + 1$, set Z/C/N flags, $\text{SC} \leftarrow 0$

10.12 DEC — Decrement

T3: $A \leftarrow \text{RF}[\text{DST}]$
 T4: $\text{RF}[\text{DST}] \leftarrow A - 1$, set Z/C/N flags, $\text{SC} \leftarrow 0$

10.13 SHL — Shift Left Logical

T3: $A \leftarrow \text{RF}[\text{DST}], n \leftarrow \text{IR}[3:0]$
 T4: $\text{RF}[\text{DST}] \leftarrow A \ll n$, set C flag if any bits shifted out, $\text{SC} \leftarrow 0$

10.14 SHR — Shift Right Logical

T3: $A \leftarrow \text{RF}[\text{DST}], n \leftarrow \text{IR}[3:0]$
 T4: $\text{RF}[\text{DST}] \leftarrow A \gg n$, set C flag if any bits shifted out, $\text{SC} \leftarrow 0$

10.15 LDI — Load Immediate

T3: $\text{RF}[\text{DST}] \leftarrow \text{zero_extend}(\text{IR}[5:0]), \text{SC} \leftarrow 0$

10.16 LDM — Load from Memory

T3: $\text{AR} \leftarrow \text{RF}[\text{SRC}]$

T4: $\text{DR} \leftarrow \text{M}[\text{AR}]$

T5: $\text{RF}[\text{DST}] \leftarrow \text{DR}, \text{SC} \leftarrow 0$

10.17 STM — Store to Memory

T3: $\text{AR} \leftarrow \text{RF}[\text{DST}]$

T4: $\text{M}[\text{AR}] \leftarrow \text{RF}[\text{SRC}], \text{SC} \leftarrow 0$

10.18 JMP — Unconditional Jump

T3: $\text{PC} \leftarrow \text{IR}[8:0], \text{SC} \leftarrow 0$

10.19 JZ — Jump if Zero

T3: if $(Z = 1)$ then $\text{PC} \leftarrow \text{IR}[5:0]; \text{SC} \leftarrow 0$

10.20 JNZ — Jump if Not Zero

T3: if $(Z = 0)$ then $\text{PC} \leftarrow \text{IR}[5:0]; \text{SC} \leftarrow 0$

10.21 HLT — Halt

T3: $\text{S} \leftarrow 0, \text{SC} \leftarrow 0$

10.22 INP — Input

T3: if $(\text{FGI} = 1)$: $\text{R0} \leftarrow \text{INPR}, \text{FGI} \leftarrow 0; \text{SC} \leftarrow 0$

10.23 OUT — Output

T3: if $(\text{FGO} = 1)$: $\text{OUTR} \leftarrow \text{R0}[7:0], \text{FGO} \leftarrow 0; \text{SC} \leftarrow 0$

10.24 SKI — Skip if Input Ready

T3: if $(\text{FGI} = 1)$: $\text{PC} \leftarrow \text{PC} + 1; \text{SC} \leftarrow 0$

10.25 SKO — Skip if Output Ready

T3: if $(\text{FGO} = 1)$: $\text{PC} \leftarrow \text{PC} + 1; \text{SC} \leftarrow 0$

10.26 ION — Enable Interrupts

T3: $\text{IEN} \leftarrow 1, \text{SC} \leftarrow 0$

10.27 IOF — Disable Interrupts

T3: $\text{IEN} \leftarrow 0, \text{SC} \leftarrow 0$

10.28 CLA — Clear R0

T3: $\text{R0} \leftarrow 0, \text{SC} \leftarrow 0$

10.29 CMR — Complement Register

T3: $A \leftarrow \text{RF}[\text{DST}]$
 T4: $\text{RF}[\text{DST}] \leftarrow \text{NOT } A$, set Z/N flags, $\text{SC} \leftarrow 0$

10.30 CLC — Clear Carry Flag

T3: $C \leftarrow 0$, $\text{SC} \leftarrow 0$

10.31 STC — Set Carry Flag

T3: $C \leftarrow 1$, $\text{SC} \leftarrow 0$

10.32 CLZ — Clear Zero Flag

T3: $Z \leftarrow 0$, $\text{SC} \leftarrow 0$

10.33 NOP — No Operation

T3: $\text{SC} \leftarrow 0$

10.34 SWAP — Swap Two Registers

T3: $\text{TR} \leftarrow \text{RF}[\text{DST}]$
 T4: $\text{RF}[\text{DST}] \leftarrow \text{RF}[\text{SRC}]$
 T5: $\text{RF}[\text{SRC}] \leftarrow \text{TR}$, $\text{SC} \leftarrow 0$

11. I/O and Interrupt Handling

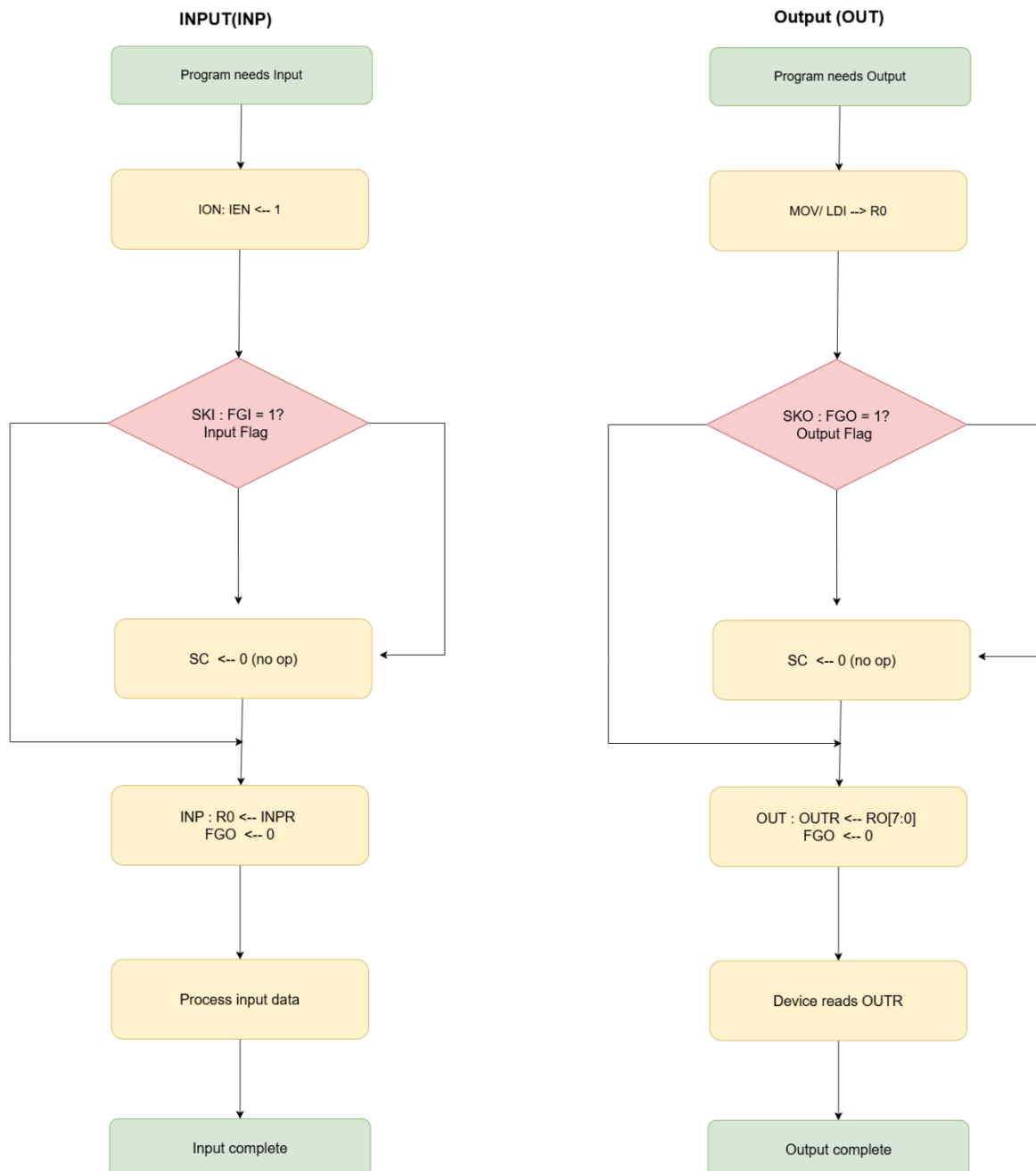
11.1 Program-Controlled I/O

VERA-16 uses program-controlled I/O. Two flag flip-flops manage data transfer readiness of FGI (Input Flag) and FGO (Output Flag). The input device sets $\text{FGI}=1$ when new data is ready in INPR. The output device sets $\text{FGO}=1$ when it is ready to accept data in OUTR. The processor polls these flags using the SKI and SKO skip instructions.

The I/O polling pattern is as follows. For input, the program carries out a SKI instruction, which skips the next instruction if $\text{FGI}=1$. If FGI is not situated then the program jumps back to retry SKI in a tight polling loop. When $\text{FGI}=1$ the INP instruction executes and transfers INPR into R0 while clearing FGI. For output the same pattern applies with SKO and OUT.

Input loop: SKI	; skip next if $\text{FGI}=1$
JMP input_loop	; if $\text{FGI}=0$, keep polling
INP	; $\text{FGI}=1$: $\text{R0} \leftarrow \text{INPR}$, $\text{FGI} \leftarrow 0$

VERA-16 Input/Output Cycle



11.2 Interrupt Mechanism

In VERA-16 when the Interrupt Enable flip-flop IEN=1 and either of the FGI=1 or FGO=1. The interrupt request flag R is set at the end of a fetch-decode-execute cycle. This prevents the interrupt cycle from corrupting an instruction in middle of an execution. Once R=1, the processor begins the interrupt cycle instead of fetching the next instruction.

Interrupt trigger condition:

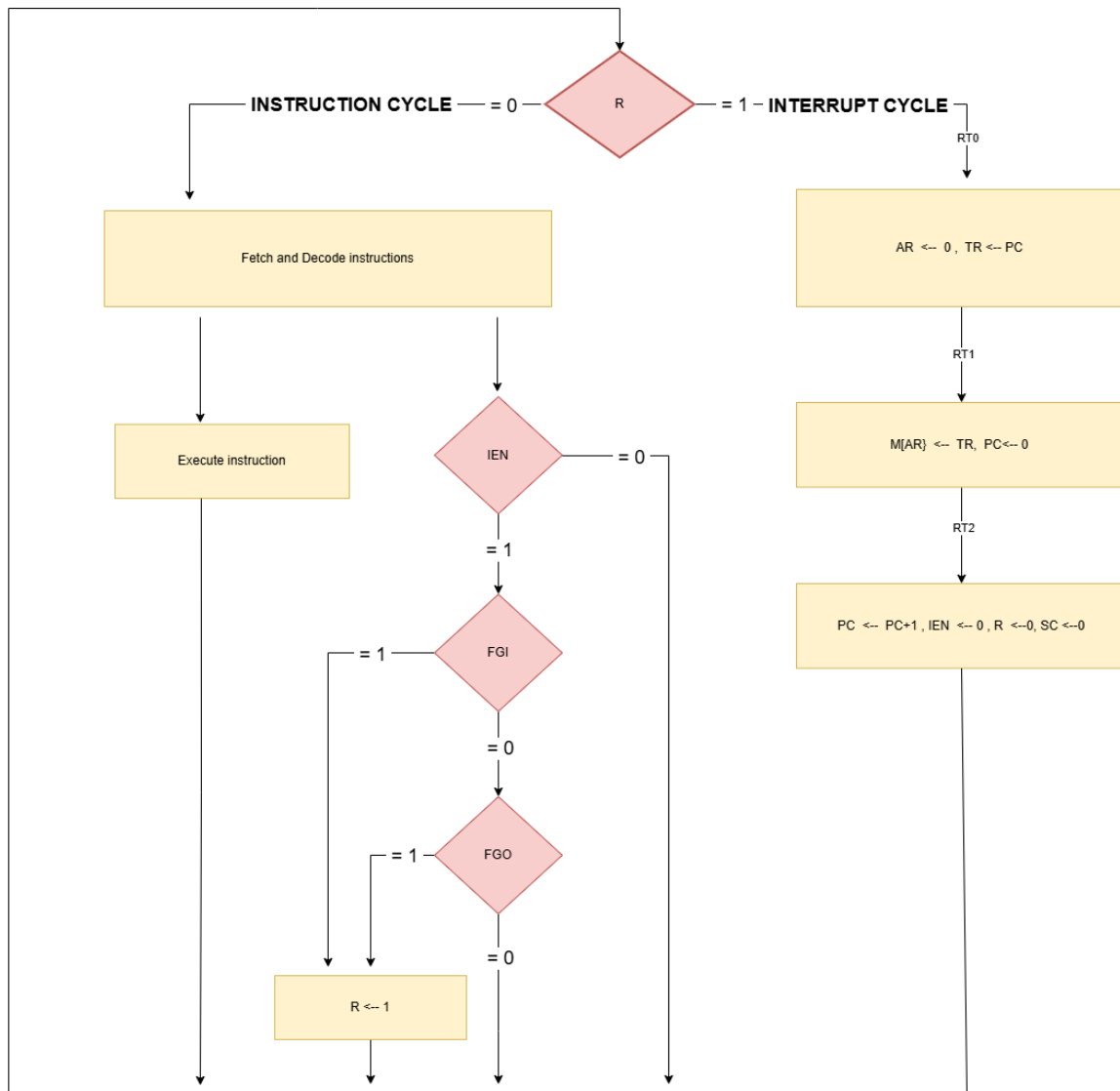
$$\text{IEN (FGI + FGO): } R \leftarrow 1$$

Interrupt cycle micro-operations:

Step	Microoperations	Purpose
RT0	$AR \leftarrow 0, TR \leftarrow PC$	Set address to 0, copy PC to TR
RT1	$M[AR] \leftarrow TR, PC \leftarrow 0$	Save PC to memory address 0
RT2	$PC \leftarrow PC + 1, IEN \leftarrow 0, R \leftarrow 0, SC \leftarrow 0$	PC now points to address 1 (ISR entry), disable interrupts, clear request flag

The primary difference in VERA-16 is that it uses R0 as the I/O data register by default (INP loads into R0, OUT reads from R0). Whereas the conventional Mano Machine uses a dedicated AC register.

VERA-16 INTERRUPT CYCLE



12. Comparison with Mano Machine

The table provides a comprehensive comparison of VERA-16 and Mano Machine. The comparison shows that VERA-16 maintains the same word size and memory capacity but internal architecture, instruction set and operational capabilities are significantly different.

Feature	Mano Machine	VERA-16	Significance
Architecture type	1-address (accumulator-based)	2-address (register file)	Removes AC bottleneck
Word size	16 bits	16 bits	Same direct comparison valid
Memory size	4096×16 RAM	4096×16 RAM	Same capacity
Instruction format	I(1) + opcode(3) + address(12)	opcode(5) + MODE(2) + DST(3) + SRC(6)	VERA-16 wider opcode space
Opcode bits	3 bits $\rightarrow$ 8 instruction slots	5 bits $\rightarrow$ 32 instruction slots	VERA-16 has 4 $\times$ more types
General-purpose registers	0 — single AC only	8 (R0–R7, 16-bit each)	VERA-16 holds more intermediates
Addressing modes	2 (direct, indirect)	4 (register, immediate, direct, reg-indirect)	VERA-16 more flexible
Instruction decoder	3-to-8 decoder	5-to-32 decoder	VERA-16 wider decode range
ALU operations	ADD, AND, complement, shift	ADD, SUB, MUL, AND, OR, XOR, NOT, SHL, SHR	VERA-16 richer arithmetic
Hardware multiplication	Not supported (software loop required)	MUL via array multiplier	VERA-16 native HW multiply
I/O method	Program-controlled via AC/INPR/OUTR	Program-controlled via R0 convention	Same method, adapted for register file
Interrupt support	IEN, FGI, FGO, RT0–RT2	IEN, FGI, FGO, RT0–RT2 (structurally identical)	Fully adapted
Sequence Counter	4-bit generates T0 - T15	3-bit generates T0 – T7	VERA-16 reduces CU hardware

13. Complete Computer Description

The VERA-16 computer is an integrated system of functional units communicating via a single 16-bit common bus. This section provides a complete data-path description.

13.1 Register File

The register file consists of 8×16 -bit general-purpose registers R0 through R7. It has two read ports (A and B) and 1 write port. Read port A is controlled by the DST field IR [8:6]; Read port B is controlled by the lower 3 bits of the SRC field IR [5:3]; the write port is also controlled by the DST field AND RegWrite control signal by CU.

13.2 Program Counter (PC)

The Program Counter is a 12-bit counter holding the address of the next instruction to fetch. It is loaded to the Address Register at T0. It is incremented by 1 at T1 immediately after the instruction fetch, so PC always points to the next sequential instruction.

13.3 Instruction Register (IR)

The Instruction Register is a 16-bit register loaded from memory at T1. At T2, the Control Unit decodes its four fields IR [15:11] (OPCODE) is sent to the 5-to-32 decoder, IR [10:9] (MODE) is sent to the 2-to-4 MODE decoder, IR[8:6] (DST) is sent to the register file selector, IR [5:0] (SRC) is interpreted according to the MODE output.

13.4 Address Register (AR)

The Address Register is a 12-bit register holding the memory address for the current operation. It is loaded from PC at T0 for instruction fetch. During LDM, it loaded from RF[*SRC*] at T3. During STM, it loaded from RF[*DST*] at T3. During the interrupt cycle, it loaded with 0 at RT0.

13.5 Data Register (DR)

The Data Register is a 16-bit temporary register for memory read results. It is loaded from M[AR] at T4 during LDM, then transferred to RF[*DST*] at T5. This two-step memory read prevents bus contention by separating the memory access cycle from the register write cycle.

13.6 ALU

The ALU is a 16-bit combinational logic block with four sub-units a ripple-carry adder, a logic unit, a barrel shifter, and an array multiplier. A 4×1 output multiplexer controlled by S3 and S2.

13.7 Control Unit

The Control Unit contains the 5-to-32 opcode decoder, the 3-bit sequence counter, the register file selector, the MODE field 2-to-4 router and the instruction decode PLA. The CU is the only source of control signals.

13.8 Memory (4096×16 RAM)

The main memory is addressed via the 12-bit Address Register AR. A read operation ($DR \leftarrow M[AR]$) or write operation ($M[AR] \leftarrow bus$) is initiated by the CU asserting the memory read/write enable. The memory data lines connect directly to the bus.

13.9 I/O System

The I/O system comprises the 8-bit INPR input and OUTR output registers, the FGI/FGO flags and the IEN interrupt enable flip-flop. The processor interacts with this system through the INP, OUT, SKI, SKO, ION, IOF instructions and the interrupt cycle.

14. Conclusion

VERA-16 fulfils its principal design motive of reducing design the accumulator bottleneck of the Mano Machine by introducing an architectural innovation of 8 general purpose registers and a 2-address instruction format. The architecture demonstrates that within the same 16-bit word as the Mano Machine, a significantly more capable instruction set is achievable.

The resulting processor supports 33 instructions across five categories, four addressing modes, hardware multiplication and a complete interrupt mechanism. The control unit is based on an architectural principal of 5-to-32 op-code decoder and a 3-bit sequence counter, providing an effective structure. The common bus system, register file and ALU word together form a coherent data flow.

The leading novelties of the VERA-16 over the Mano Machine are the general-purpose register file, the 2-address instruction format, 5-bit op-code, four addressing modes and the array multiplier.

The current design has some limitations such as a restricted immediate value range (0–63) due to the 6-bit SRC field, the limited direct addressing range (63 memory locations) and the 16-bit product abbreviation for MUL. Future add-ons could address these issues by increasing the size of the instruction word to 24 or 32 bits and introducing a dedicated branch offset fields or implementing a hardware division unit.

Bibliography

- Morris Mano, Computer System Architecture, 3rd Edition, Prentice Hall, 1993.
- David A. Patterson and John L. Hennessy, Computer Organization and Design: The Hardware/Software Interface, 5th Edition, Morgan Kaufmann, 2014.
- Prof. Dr. G. Abbas (GIK Institute), CE-222 Computer Organization and Assembly Language, Lecture Slides Lectures 14–22.
- Draw.io Diagram Tool, <https://app.diagrams.net>.
- William Stallings, Computer Organization and Architecture: Designing for Performance, 10th Edition, Pearson, 2016.